Deep Neural Networks for Survival Analysis with survdnn

Architecture, loss functions, and prediction in a native R torch implementation.

Imad EL BADISY

2026-08-19

Table of contents

Right-censored survival data $\{(t_i, \delta_i, x_i)\}_{i=1}^n$ pair an observed time $t_i = \min(T_i, C_i)$ and an event indicator $\delta_i = \mathbb{1}(T_i \leq C_i)$ with a covariate vector $x_i \in \mathbb{R}^p$. The Cox proportional hazards model expresses the hazard as $\lambda(t \mid x) = \lambda_0(t) \exp(\beta^\top x)$, restricting covariate effects to a linear, time-constant log-hazard ratio. `survdnn` replaces the linear predictor $\beta^\top x$ with a multilayer perceptron $f_\theta(x)$, keeping the rest of the estimation machinery (partial likelihood, Breslow baseline, time-dependent risk) intact, and extends this replacement to non-proportional-hazards and accelerated failure time formulations.[1] The package is implemented natively in R on top of `torch`, avoiding a Python backend.[2]

[1] `survdnn` package documentation, CRAN: https://CRAN.R-project.org/package=survdnn.

[2] EL BADISY, I. (2025). *SurvDNN: Survival Deep Learning Models for Tabular Data.* Submitted to The R Journal.

Architecture

For an input of dimension p and hidden layer sizes $h_1, \dots, h_L$, the network is a sequential composition of blocks

$$z^{(0)} = x, \qquad z^{(l)} = \phi\big(\mathrm{BN}\big(W^{(l)} z^{(l-1)} + b^{(l)}\big)\big), \qquad l = 1, \dots, L,$$

where ϕ is one of `relu`, `leaky_relu`, `tanh`, `sigmoid`, `gelu`, `elu`, or `softplus`, BN is an optional batch normalization layer, and dropout with rate p_{drop} is applied after each block. The output layer is a single linear map $g_\theta(z^{(L)}) = W^{(L+1)} z^{(L)} + b^{(L+1)} \in \mathbb{R}$, so the whole network is $f_\theta(x) = g_\theta(z^{(L)}(x))$. Covariates are standardized before entering the network, and the standardization constants are stored on the fitted object and reapplied at prediction time.

Interpretation of the scalar output $f_\theta(x)$ depends on the loss used to train it: a log-risk score under Cox losses, a location parameter on the log-time scale under AFT, or a time-dependent risk score under Cox-Time.

Loss functions

Cox partial likelihood

For a batch sorted by descending observed time, the risk set at time t_i is $R(t_i) = \{j : t_j \geq t_i\}$. `survdnn` trains f_θ against the negative partial log-likelihood

$$\ell_{\text{cox}}(\theta) = -\frac{1}{n_e} \sum_{i:\, \delta_i = 1} \left(f_\theta(x_i) - \log \sum_{j \in R(t_i)} \exp f_\theta(x_j) \right),$$

where n_e is the number of events. The implementation computes the inner sum with `torch_logcumsumexp` on time-sorted linear predictors, so risk-set membership falls out of the sort order rather than an explicit double loop. An L2 variant adds $\lambda \cdot \overline{f_\theta(x)^2}$ to this loss, penalizing the magnitude of the risk score directly rather than the network weights.

Accelerated failure time (log-normal)

Under the AFT loss, the network output $\mu_\theta(x)$ is the location parameter of a log-normal model for the event time: $\log T = c + \mu_\theta(x) + \sigma Z$, $Z \sim \mathcal{N}(0, 1)$, where c is a location offset fixed at the mean log event-time in the training data (stored as `aft_loc`) to keep the residual $\log t - c$ near zero. The scale σ is a single learnable parameter, optimized jointly with θ as $\exp(\log \sigma)$ for positivity. The censored negative log-likelihood combines an exact-density term for events and a survival term for censored observations,

$$\ell_{\text{aft}}(\theta,\sigma) = \frac{1}{n}\sum_{i=1}^{n}\Big[\delta_i\big(\log t_i + \log\sigma + \tfrac{1}{2}z_i^2\big) - (1-\delta_i)\log S_Z(z_i)\Big], \qquad z_i = \frac{\log t_i - c - \mu_\theta(x_i)}{\sigma},$$

with $S_Z(z) = 1 - \Phi(z)$ the standard normal survival function, evaluated through the error function for numerical stability. This is the standard parametric censored-data likelihood, with the location term replaced by a network.

Cox-Time

The Cox-Time loss extends the Cox partial likelihood to a time-varying log-risk function $g_\theta(t, x)$, taking time as an explicit network input rather than as a fixed offset.[3] The contribution of an event at t_i becomes

$$\ell_{\text{coxtime}}(\theta) = -\frac{1}{n_e}\sum_{i:\,\delta_i=1}\left(g_\theta(t_i, x_i) - \log\sum_{j\in R(t_i)}\exp\, g_\theta(t_i, x_j)\right),$$

which differs from the Cox loss only in that the denominator re-evaluates the network at t_i for every subject at risk, rather than reusing a single risk score per subject. This makes the loss non-separable in `pred` alone: it needs the network and the full covariate matrix to build the $g_\theta(t_i, x_j)$ grid at every event time. The exported `coxtime_loss(pred, true)` reflects this directly, it raises an informative error rather than silently computing something incorrect, and the working implementation is the internal factory `survdnn__coxtime_loss_factory(net)`, which `survdnn()` uses when `loss = "coxtime"`. Risk-set membership is computed on raw observed time, while the time fed to the network is centered and scaled for training stability.

Training

Fitting proceeds by standard gradient-based optimization: for each epoch, a forward pass computes the loss on the full training set, `backward()` accumulates gradients, and one of `adam`, `adamw`, `sgd`, `rmsprop`, or `adagrad` updates the parameters (`adam`/`adamw` default to a small weight decay of `1e-4` unless overridden). Training is full-batch rather than mini-batch, so an epoch is one optimizer step over the entire dataset. Callbacks receive the epoch index and current loss after each step and can halt training early, as with `callback_early_stopping()`. Missing values in the model variables are either dropped (`na_action = "omit"`) or treated as an error (`na_action = "fail"`) before any tensor is built.

[3] Kvamme, H., Borgan, Ø., & Scheel, I. (2019). Time-to-event prediction with neural networks and Cox regression. *Journal of Machine Learning Research*, 20(129), 1-30.

Prediction

`predict.survdnn()` returns one of three things, depending on `type`. `type = "lp"` returns the raw network output, interpreted according to the training loss. `type = "survival"` returns predicted survival probabilities at a set of times, and `type = "risk"` returns $1 - S(t)$ at a single time.

For Cox and Cox-L2 models, survival curves are not read directly off the network; they combine the fitted risk score with a Breslow estimate of the cumulative baseline hazard $H_0(t)$, obtained by refitting a one-covariate `coxph()` on the training linear predictors:

$$\hat{S}(t \mid x) = \exp(-\widehat{H}_0(t)\exp(f_\theta(x))).$$

For the AFT model, survival probabilities follow directly from the log-normal form, $\hat{S}(t \mid x) = 1 - \Phi\big(\frac{\log t - c - \mu_\theta(x)}{\sigma}\big)$, using the same c and learned σ stored at training time.

For Cox-Time, baseline hazard increments are estimated at each observed event time t_k from the training risk sets, $d\widehat{H}_0(t_k) = dN(t_k) \big/ \sum_{j \in R(t_k)} \exp\, g_\theta(t_k, x_j)$, and cumulative hazard at a query time is obtained by summing increments at all event times up to that point, evaluated with the network at each of them: $\widehat{H}(t \mid x) = \sum_{t_k \le t} \exp\, g_\theta(t_k, x)\, d\widehat{H}_0(t_k)$. This requires one forward pass per event time per subject, which is the computational cost of allowing the risk score to vary with time.

Evaluation

Three metrics are implemented directly against predicted survival matrices rather than delegated to an external package: the concordance index (`cindex_survmat()`), evaluated at a fixed horizon by comparing $1 - S(t^\star \mid x)$ across all admissible pairs; the IPCW-weighted Brier score (`brier()`) at a single time point, using a Kaplan-Meier estimate of the censoring distribution as weights; and the integrated Brier score (`ibs_survmat()`), a trapezoidal integral of the Brier score over a grid of times, normalized by the width of the grid. `evaluate_survdnn()` wraps these for a fitted model, and `cv_survdnn()` repeats fit-and-evaluate over stratified folds via `rsample::vfold_cv()`, stratifying on the event indicator so that fold sizes stay balanced.

Example

A model is fit with the same formula interface as `survival::coxph()`.

```r
library(survdnn)
library(survival)

veteran <- survival::veteran

mod <- survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data       = veteran,
  hidden     = c(16, 8),
  loss       = "cox",
  epochs     = 100,
  lr         = 1e-3,
  .seed      = 1,
  verbose    = FALSE
)

mod
```

Survival curves at a grid of times follow the same `predict()` interface used for `coxph` and `survfit` objects.

```r
pred <- predict(mod, newdata = veteran, type = "survival", times = c(30, 90, 180))
round(head(pred), 3)
```

```
   t=30  t=90 t=180
1 0.806 0.573 0.259
2 0.847 0.652 0.354
3 0.835 0.629 0.325
4 0.832 0.622 0.316
5 0.844 0.645 0.346
6 0.684 0.376 0.093
```

Model assessment

Discrimination and calibration are read off the same predicted matrix.

```r
y <- model.response(model.frame(mod$formula, veteran))

cindex_survmat(y, pred, t_star = 180)
```

```
 c-index
0.733631
```

```
ibs_survmat(y, pred, times = c(30, 90, 180))
```

```
     ibs
0.177018
```

Cross-validated performance follows the same call structure, now with the model refit per fold.

```
res <- cv_survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data    = veteran,
  times   = c(30, 90, 180),
  metrics = c("cindex", "ibs"),
  folds   = 3,
  .seed   = 42,
  hidden  = c(16, 8),
  epochs  = 50,
  verbose = FALSE
)

summarize_cv_survdnn(res)
```

```
# A tibble: 2 x 5
  metric  mean      sd lower upper
  <chr>  <dbl>   <dbl> <dbl> <dbl>
1 cindex 0.601 0.0515  0.542 0.659
2 ibs    0.209 0.00615 0.202 0.216
```

Practical notes

The AFT and Cox-Time losses are more expensive per epoch than plain Cox: AFT adds one learnable global parameter, and Cox-Time evaluates the network on a full $n_e \times n$ grid of (event time, subject) pairs at every step, so training cost grows with the number of events rather than staying constant. Batch normalization interacts with very small batches; since `survdnn` trains full-batch by default, this is only a concern for small datasets, where turning batch normalization off (`batch_norm = FALSE`) is often more stable than leaving it on. Dropout and weight decay are the two regularizers exposed directly; `tune_survdnn()` and `gridsearch_survdnn()` search over these together with architecture and learning-rate settings under cross-validation.